

Технически университет, София
Факултет по компютърни системи и технологии
Курсов проект по „Сигурни протоколи и архитектури за комуникации“

Разпознаване на TLS и некриптиран HTTP по първия октет при един слушащ сокет

Механизъм и анализ на сигурността

Алекс Цветанов, група 263
Ръководител: проф. Георги Илиев

github.com/Alex-Tsvetanov/octet

София, 2026

Два порта или един

Обичайно

порт 80: некриптиран HTTP

порт 443: HTTP над TLS

Всеки порт знае предварително какво ще пристигне.

В този проект

един слушащ сокет

за всяка връзка сървърът сам решава: TLS или HTTP?

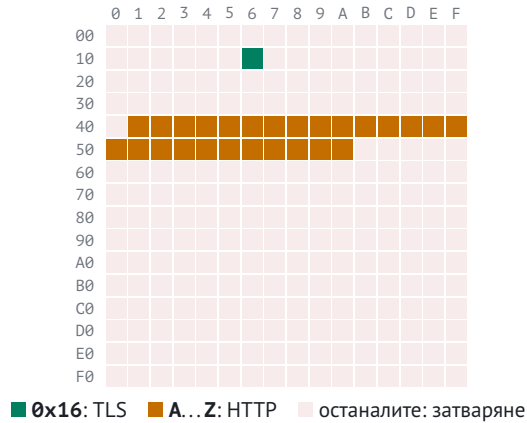
- Решението се взема преди TLS ръкостискането, когато нищо още не е защитено.
- Въпросите: стига ли един октет, губят ли се данни, какво струва, какво се отваря за нападателя.

Защо един октет е достатъчен

TLS RFC 8446, раздел 5.1
Клиентът говори пръв. Първият запис е ClientHello, ContentType **`22 = 0x16`**.

HTTP RFC 9110, раздел 9.1
Всеки метод е с главни латински букви: `GET`, `POST`, … Префиксът на HTTP/2 започва с `PRI`. RFC 9113, раздел 3.4

Двете множества не се пресичат. Тестът минава през всичките 256 стойности (538 проверки).



Разпознаването в кода

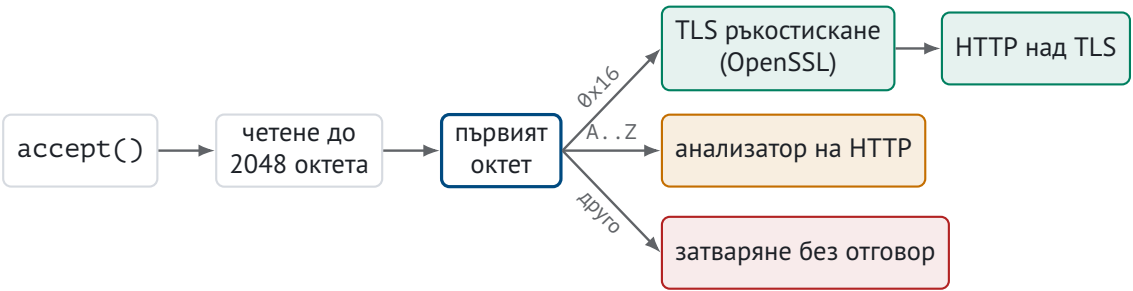
```
36  [[nodiscard]] constexpr Wire classify(std::span
      <const std::uint8_t> first) noexcept
37  {
38    if (first.empty())
39    {
40      return Wire::Unknown;
41    }
42    const std::uint8_t octet = first.front();
43    if (octet == kContentHandshake)
44    {
45      return Wire::Tls;
46    }
47    if (octet >= 'A' && octet <= 'Z')
48    {
49      return Wire::Cleartext;
50    }
51    return Wire::Unknown;
52  }
```

include/octet/classify.hpp

- Гледа само първия октет.
- `constexpr` и `noexcept`: без памет, без изключения, проверимо при компилиране.
- Празен вход дава „непознато“, а не предположение.

Пътят на една връзка

един общ краен срок: от `accept()` до края на ръкостискането



- Нишка за всяка връзка, с горна граница на едновременните връзки.
- Непознатото се затваря мълчаливо: отговорът би дал информация на сканиращия.

Прочетените октети не се губят

`MSG_PEEK`
Октетите остават в ядрото, следващото четене ги получава отново. Цена: второ системно извикване за същите данни.

Връщане (`ReplayReader`) избраният начин
Октетите се четат наистина и се подават първи на TLS слоя, после се чете от сокета.

- OpenSSL работи върху два буфера в паметта (`BIO_s_mem`), а не върху сокета.
- `BIO_set_mem_eof_return(-1)`: празен буфер означава „опитай пак“, не „край на потока“.
- Всички 2000 измерени TLS връзки с разпознаване: първото четене донася целия ClientHello, 517 октета. Няма допълнително системно извикване.

Един абсолютен краен срок

Относителен срок, който започва отначало при всяко четене, не спира клиент, който праща по един октет точно преди изтичането. Той държи връзката отворена неограничено.

Срокът в `octet` е момент във времето, зададен при приемането. Той покрива разпознаването **и** цялото ръкостискане.

Клиент, който праща **`0x16`** и спира, получава същия срок като клиент, който не праща нищо.

```
27    [[nodiscard]] int remaining_ms() const
      noexcept
28    {
29      const auto left = at_ - Clock::now();
30      if (left <= Clock::duration::zero())
31      {
32        return 0;
33      }
34      const auto ms = std::chrono::ceil<std::
      chrono::milliseconds>(left).count();
35      return static_cast<int>(std::min<long
      long>(ms, 24LL * 60 * 60 * 1000));
36    }
```

include/octet/deadline.hpp

Настройки на TLS

Настройка	Причина
минимум TLS 1.2, по избор 1.3	по-старите версии не се договарят RFC 9325, 3.1.1
без предоговаряне	атаки върху предоговарянето в TLS 1.2 RFC 5746; RFC 7457, 2.10
без компресия	отваря атаки като CRIME RFC 7457, 2.6; RFC 9325, 3.3
сървърът избира шифъра	не клиентът, от общите шифри
строг ALPN	без общ протокол: фатално предупреждение 120 RFC 9325, 3.8
ключът спрямо сертификата	грешката спира сървъра при стартиране, не при първия клиент

Тестов удостоверяващ орган, ECDSA P-256: CA и сертификат на сървъра за `localhost`. Клиентът проверява веригата и името, без да се изключва нищо.

Отказ по ALPN, видян в мрежата

Клиент, чийто списък с протоколи няма `http/1.1`, получава преди ServerHello, некриптирано:



- `no_application_protocol` RFC 8446, раздел 6; RFC 7301, раздел 3.2
- Интеграционният тест сравнява точно тези октети, а не текста на грешката. На Linux Python с OpenSSL 3.0.13 не може дори да назове предупреждението.

Заплахи и отговори

Заплаха	Отговор в кода	Проверено
свързва се и мълчи	общ краен срок от приемането	тест
праща **`0x16`** и спира	същият срок покрива ръкостискането	тест
по един октет преди изтичане	срокът е абсолютен	по построение
много мълчащи връзки	граница; новите се затварят	тест
`0x16` и безсмислени данни	OpenSSL отхвърля записа	тест
двоични данни, `get`	„непознато“, без отговор	тестове
огромна заглавна част	граница 8192 октета, отговор 431	тест
стара версия на TLS	минимална версия на контекста	тест

Разпознаването **насочва**, а не валидира. Проверките на записа ги прави OpenSSL.

Грешката, която тестовете откриха

Първата версия проверяваше границата от 8192 октета само *докато търси* празния ред, който завършва заглавната част.

Едно четене донася и октетите над границата, и празния ред: празният ред се намира пръв и заявката се приема.

Поправка: позицията на празния ред се сравнява с границата.

тест на функция: заявката по един октет
минава

интеграционен тест: поле от 9000 октета
200 вместо 431

нов тест на функция: заявката на големи части

Тестът на функция не можеше да я открие, защото не доставя данните така, както мрежата.

Проверка

Тест	проверки
разпознаване, 256 стойности	538
четене с връщане	12
анализатор на HTTP	22
TLS сесия в паметта	47
интеграционен, истински сокети	35

- Минават на **Windows 11** (MSVC 19.51, OpenSSL 3.6.4) и на **Ubuntu 24.04 във WSL2** (GCC 14.2, OpenSSL 3.0.13).
- Без предупреждения при `/W4` и при `-Wall -Wextra -Wpedantic`.
- macOS: конфигурация има, проверка няма.

Измерване: постановка

Четири варианта, всеки като отделен процес на една машина:

	с разпознаване	отделен слушател
TLS	един сокет, решение по октета	ръкостискане веднага
HTTP	един сокет, решение по октета	направо към анализатора

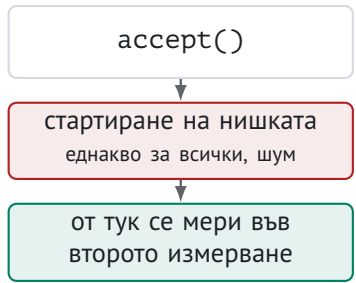
- Една заявка на връзка. Клиент на Python, 100 връзки в секунда, проверка на сертификата и името.
- 5 повторения по 400 връзки на вариант, ред на вариантите разбъркан (начално число 20261103).
- Ефект = разлика на медианите; 95% интервал с повторна извадка, 2000 пъти, в рамките на всяко повторение.
- Минути, а не часове: Windows 273 s, Linux 101 s.

Първото измерване мереше друго

Времената се мереха от `accept()`. Разликите излязоха 280,0 µs за TLS и 83,5 µs за HTTP.

Но при HTTP с отделен слушател времето до решението беше **0,396 ms**, а този вариант не чете нищо преди решението.

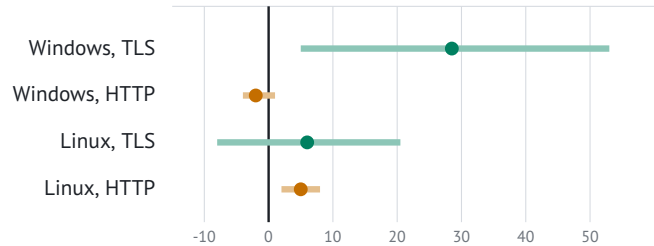
Мереше се стартирането на нишката, не разпознаването.



Първото измерване е запазено в хранилището непроменено, като доказателство за проблема.

Резултат

Ефект на разпознаването, от стартирането на нишката (µs, 95% интервал)



В паметта
Windows 211,5 ns, Linux 163,6 ns на връзка.

TLS връзката
3,108 ms на Windows, 2,163 ms на Linux.

Цената е под една хилядна от връзката и под разделителната способност от край до край.

Linux, HTTP: 5,0 µs с интервал без нулата. Причината не е установена и числото не се приписва на разпознаването.

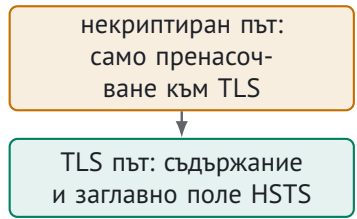
Сериозното ограничение не е в кода

Съчетаването на некриптиран и TLS трафик отваря пътя за SSL Stripping: нападател по средата премахва преминаването към TLS. RFC 9325, 3.2; RFC 7457, 2.1

Разпознаването не създава проблема (той съществува и при два порта), но прави некриптирания достъп възможен по подразбиране.

`octet` сега: едно и също съдържание и по двата пътя, без HSTS.

Правилната схема



HSTS: RFC 6797. Ползата от един порт остава.

Не е правено: fuzzing, одит от трета страна. Нишка на връзка не издържа на голяма атака за отказ от обслужване.

Изводи

- Един октет разделя TLS от HTTP за всичките 256 стойности. Основата е в стандартите.
- Реализацията не губи данни и не отслабва TLS: връщане на октетите, общ краен срок, строг ALPN.
- Цената е под една микросекунда на връзка. Производителността не е аргумент срещу механизма.
- Аргументът, който остава, е сигурността: пренасочване и HSTS, иначе схемата е уязвима за SSL Stripping.

Следващи стъпки: пренасочване и HSTS; fuzzing на анализатора и на разпознаването; измерване на физически Linux през истинска мрежа.

Въпроси?

`github.com/Alex-Tsvetanov/octet`

Кодът, тестовете, суровите записи от измерванията и документът

Резерва: Windows, от стартирането на нишката (ms)

Рамо	*n*	p50	p90	p99	медиани по повторения
TLS, с разпознаване	2000	3,137	6,964	20,840	3,088 … 3,236
TLS, отделен слушател	2000	3,108	6,254	22,496	3,076 … 3,154
HTTP, с разпознаване	2000	0,126	0,436	20,424	0,124 … 0,128
HTTP, отделен слушател	2000	0,128	0,502	18,616	0,117 … 0,136

Последната колона: най-малката и най-голямата медиана сред повторенията. При TLS разликата между тях е по-голяма от самите ефекти.

Резерва: Linux във WSL2, от стартирането на нишката (ms)

Рамо	*n*	p50	p90	p99	медиани по повторения
TLS, с разпознаване	2000	2,169	13,636	27,076	2,084 … 4,008
TLS, отделен слушател	2000	2,163	10,390	16,126	2,109 … 9,543
HTTP, с разпознаване	2000	0,084	0,116	0,242	0,072 … 0,095
HTTP, отделен слушател	2000	0,079	0,110	0,183	0,068 … 0,090

Ограничения: една машина, без мрежа; клиент на Python, по-бавен от сървъра; Linux във виртуална машина; нишка на връзка.

Резерва: отклонение от RFC 9112

RFC 9112, раздел 2.2: сървърът *SHOULD* да игнорира празен ред преди реда на заявката.

`octet` не го прави: връзка, която започва с **`CR (0x0D)`**, се затваря като непозната.

Причина: иначе решението не може да се вземе по първия октет и трябва да се чака края на празните редове. Изборът е съзнателен и е описан в документа.